

Báo cáo Kiểm tra và Nghiệm thu

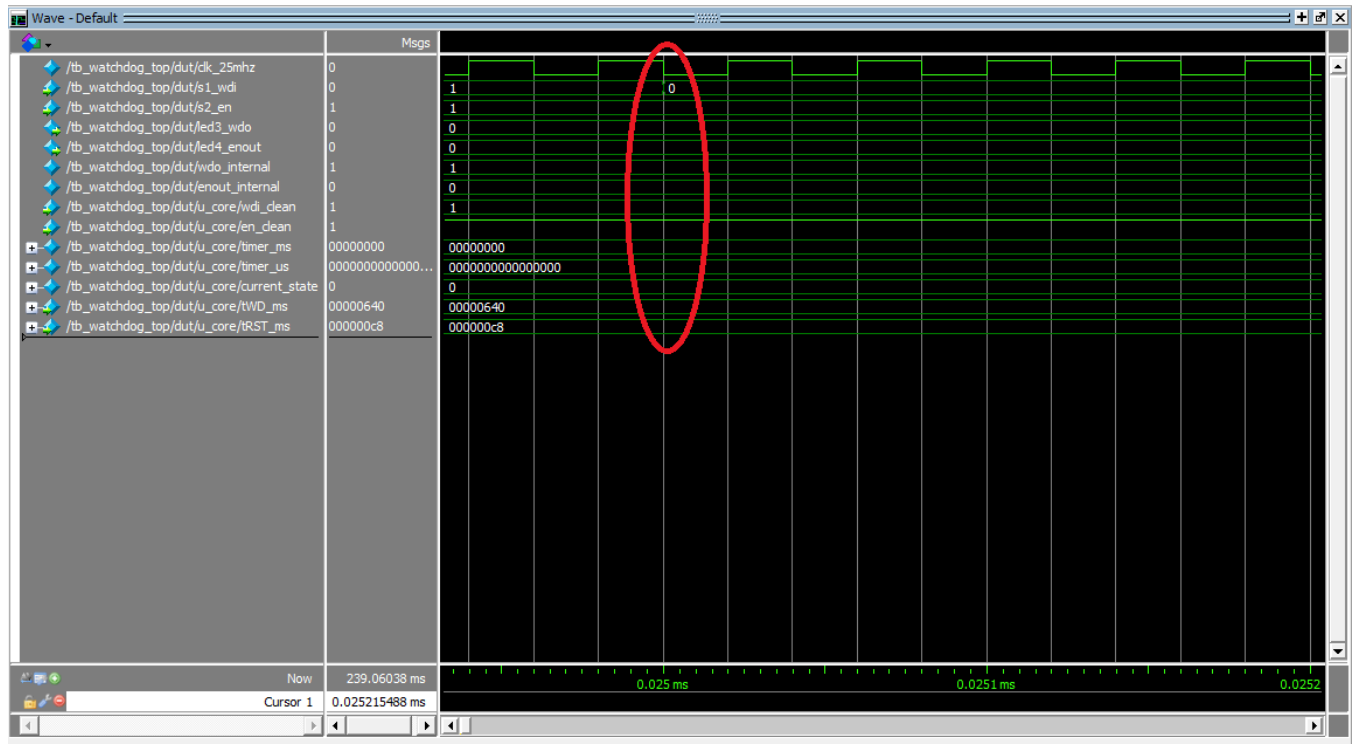
Nhóm: IC824 - Trường Điện Điện Tử - HUST

I. Danh mục Verification Checklist và Vị trí thực thi mã nguồn

STT	Mục	Nội dung Yêu cầu (Theo tài liệu đặc tả)	Giải pháp & Vị trí Code	Waveform
1	4.1	1. Sau reset hệ thống, watchdog ở trạng thái disable.	Gán <code>current_state = ST_IDLE</code> , ngõ ra an toàn (<code>WDO=1</code> , <code>ENOUT=0</code>). File watchdog_fsm_core.v (Dòng 43 và 73-76).	Hình 4.1.2
		2. <code>EN=0</code> : watchdog disable, <code>WDI</code> bị bỏ qua; <code>ENOUT=0</code> ; <code>WDO</code> nhả.	Ép <code>next_state = ST_IDLE</code> , chốt cứng ngõ ra <code>WDO=1</code> , <code>ENOUT=0</code> File watchdog_fsm_core.v (Dòng 56 và 73-76)	
		3. <code>EN</code> chuyển 0-1: đếm <code>arm_delay_us</code> , bỏ qua kick <code>WDI</code> .	Chuyển <code>ST_ARMING</code> . Kích hoạt <code>timer_us</code> , giữ <code>ENOUT=0</code> và không có logic xử lý <code>final_kick</code> ở trạng thái này. File watchdog_fsm_core.v (Dòng 59 và 85-89).	Hình 4.1.3
		4. Sau <code>arm_delay_us</code> : đếm <code>tWD</code> ; <code>ENOUT=1</code> .	Chuyển <code>ST_MONITOR</code> khi đủ <code>arm_delay_us</code> . Kéo <code>ENOUT=1</code> và bắt đầu đếm <code>timer_ms</code> . File watchdog_fsm_core.v (Dòng 60 và 90-94).	Hình 4.1.4
2	4.2	5. Kick hợp lệ là 'falling edge' của <code>WDI</code> .	Dùng thanh ghi trễ <code>wdi_clean_d</code> bắt mẫu trạng thái từ 1 xuống 0 để tạo <code>wdi_falling_edge</code> . File watchdog_fsm_core.v (Dòng 34-36)..	Hình 4.2.5
		6. Mỗi kick hợp lệ reset timer <code>tWD</code> về 0.	Tại trạng thái <code>ST_MONITOR</code> , khi có <code>final_kick</code> , thực hiện gán <code>timer_ms <= 32'd0</code> . File watchdog_fsm_core.v (Dòng 92)	
	4.2	7. <code>tWD</code> hết hạn trước khi có kick: <code>assert WDO=0</code> trong <code>tRST_ms</code> ; sau đó nhả <code>WDO</code> và bắt đầu chu kỳ mới.	Chuyển <code>ST_FAULT</code> khi <code>timer_ms >= tWD_ms</code> (ép <code>wdo_logic <= 0</code>). Đếm đủ <code>tRST_ms</code> thì tự động chuyển về <code>ST_MONITOR</code> (nhả <code>wdo_logic <= 1</code>). File watchdog_fsm_core.v (Dòng 63-67 và 95-99).	Hình 4.2.7
		8. Cho phép ghi <code>CLR_FAULT</code> để nhả <code>WDO</code> ngay.	Tại logic chuyển trạng thái của <code>ST_FAULT</code> , bổ sung điều kiện <code> ctrl_reg[2] == 1'b1</code> để lập tức quay về <code>ST_MONITOR</code> mà không cần đợi hết <code>tRST_ms</code> .	Hình 4.2.8

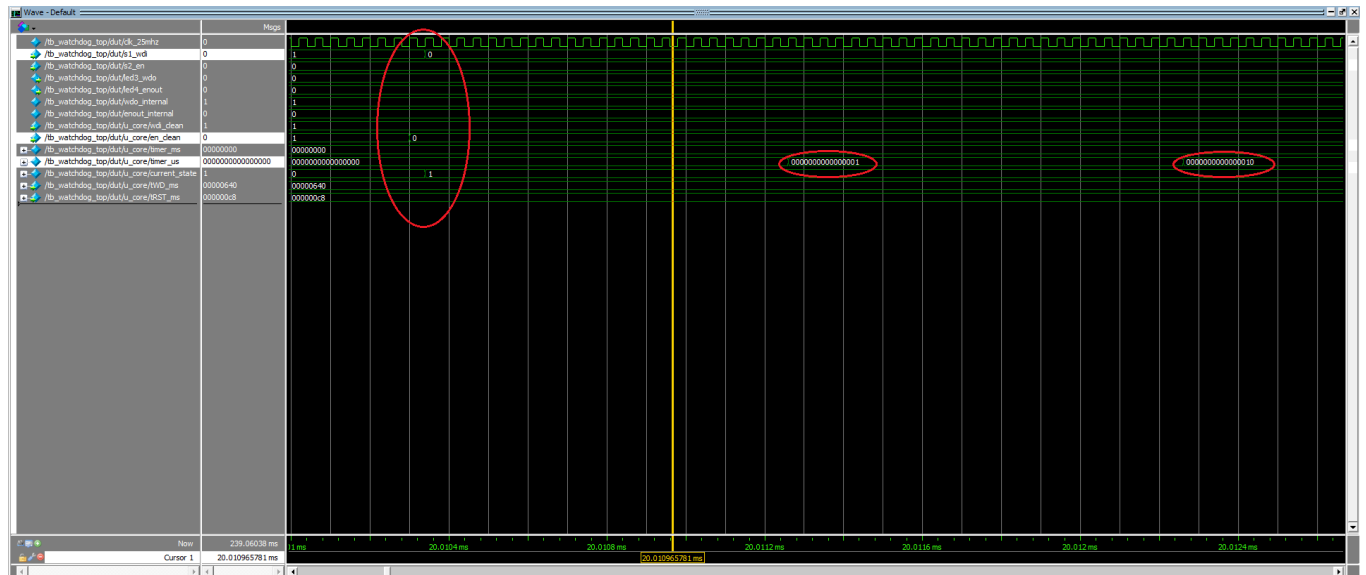
			File <u>watchdog_fsm_core.v</u> (Dòng 66-67).	
3	4.3	Xuất tín hiệu push-pull nhưng vẫn giữ quy ước active-low cho WDO và logic hợp lệ cho ENOUT.	Xuất tín hiệu dạng Push-Pull qua lệnh assign. Thực hiện đảo logic ~wdo_logic để tương thích phần cứng File <u>io_buffers.v</u> (Dòng 16 và 20)	Hình 4.3
4	5.1	Frame: [0x55][CMD][ADDR][LEN][DATA...][CHK].	File: <u>regfile_and_parser.v</u> (Dòng 71 - 103)	Hình 5
		CHK = XOR tất cả byte từ CMD đến hết DATA.	File <u>regfile_and_parser.v</u> (Dòng 74, 84, 89, 99 và 104).	
		Trả ACK/RESP qua UART (tối thiểu trả STATUS hoặc echo).	Lệnh Write và Kick trả ACK tx_data <= 8'hAA. Lệnh Read trả data qua bộ đệm. Lệnh Get_Status trả tx_data <= status_in[7:0]. File <u>regfile_and_parser.v</u> (Dòng 113, 133, 139 và 157)	
5	5.2	Ánh xạ 5 thanh ghi: 0x00 CTRL, 0x04 tWD_ms, 0x08 tRST_ms, 0x0C arm_delay_us, 0x10 STATUS đúng độ rộng bit và tính chất R/W, R như mô tả.	File <u>regfile_and_parser.v</u> (Dòng 25-28, 107-113, và 119-127).	Hình 5.2.1 Hình 5.2.2 Hình 5.2.3 Hình 5.2.4 Hình 5.2.5 Hình 5.2.6 Hình 5.2.7 Hình 5.2.8 Hình 5.2.9 Hình 5.2.10
6	5.3	Hỗ trợ: 0x01 WRITE_REG, 0x02 READ_REG, 0x03 KICK, 0x04 GET_STATUS.	File: <u>regfile_and_parser.v</u> (Dòng 106, 118, 131 và 138).	Hình 5.3.1 Hình 5.3.2 Hình 5.3.3 Hình 5.3.4
7	6	Mapping chân và tài nguyên trên board WDI (Button S1) EN (Button S2) WDO (LED D3) ENOUT (LED D4),.	File <u>Watchdog.cst</u>	

II. Kết quả thực nghiệm mô phỏng



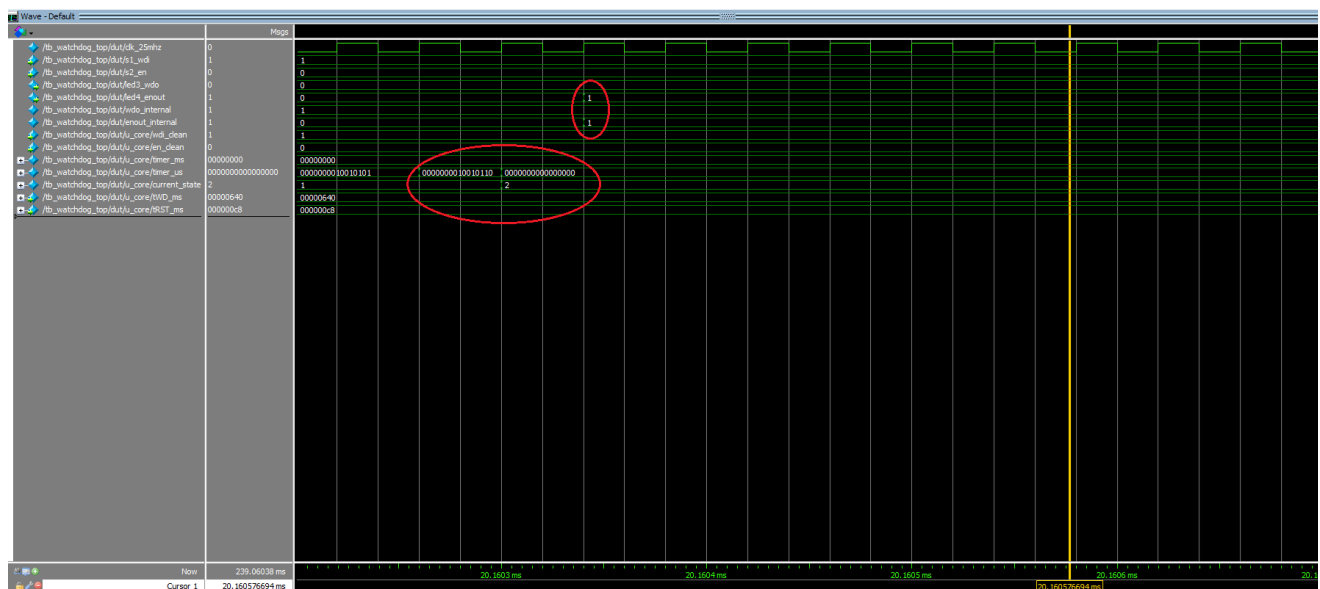
Hình 4.1.2. Mô phỏng logic bỏ qua tín hiệu WDI khi chưa Enable tại mốc 0.025 ms

Tại mốc 0.025 ms, waveform chứng minh Mục 2: Khi hệ thống chưa được kích hoạt ($s2_en = 1$, $en_clean = 0$), một xung tác động thử nghiệm được đưa vào chân $s1_wdi$ (khoanh đỏ) nhưng máy trạng thái $current_state$ vẫn đứng yên ở mức 0 và các bộ đếm timer không hoạt động. Watchdog bỏ đi mọi tín hiệu kick giả mạo nếu chưa được Enable chính thức



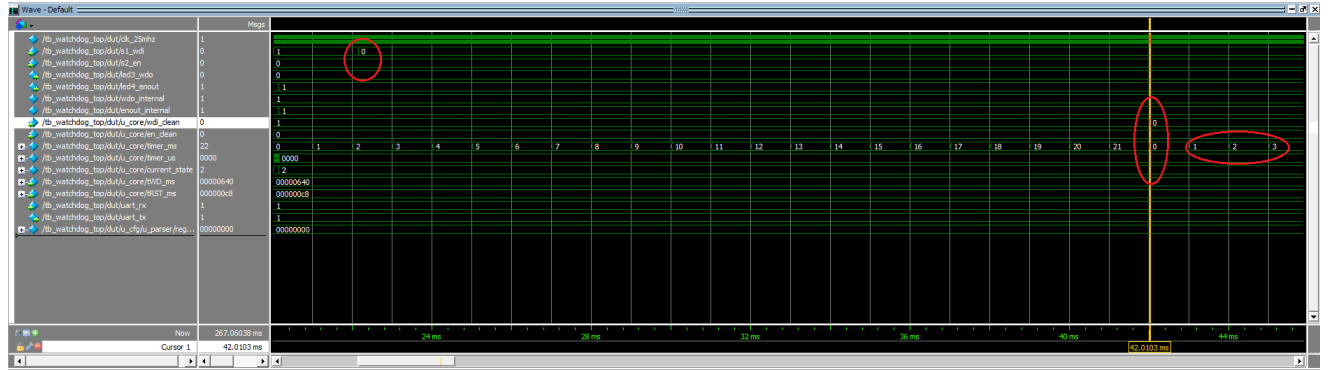
Mốc 20.01 ms: Tín hiệu en_clean xác lập mức tích cực (1 -> 0). Sau 2 chu kỳ clock, FSM chuyển từ ST_IDLE (0) sang ST_ARMING (1) và bắt đầu đếm timer_us. Xung Kick WDI ngay sau đó bị hệ thống lờ đi, không làm reset bộ đếm.

Mốc 20.01 ms: Tín hiệu en_clean xác lập mức tích cực (1 -> 0). Sau 2 chu kỳ clock, FSM chuyển từ ST_IDLE (0) sang ST_ARMING (1) và bắt đầu đếm timer_us. Xung Kick WDI ngay sau đó bị hệ thống lờ đi, không làm reset bộ đếm.



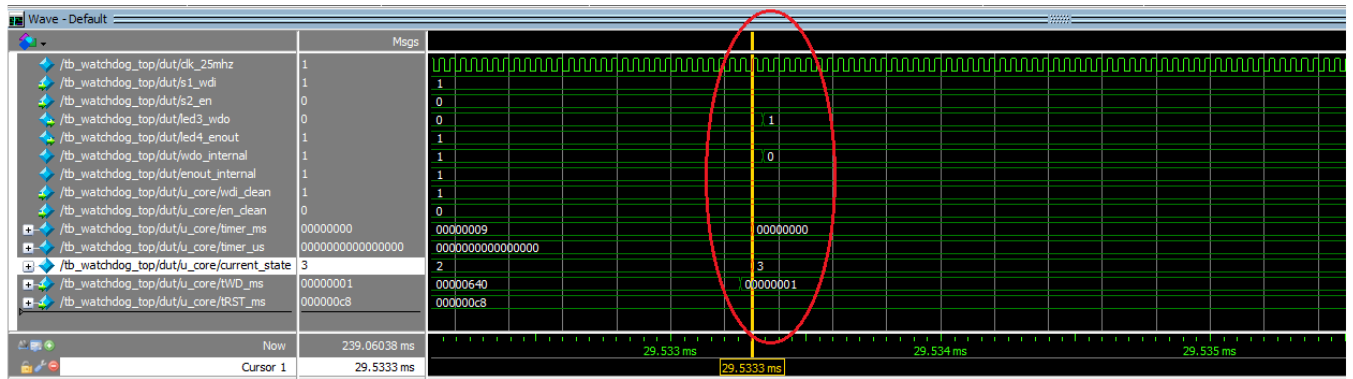
Hình 4.1.4. Dạng sóng quá trình FSM chuyển sang ST_MONITOR, ngõ ra ENOUT xác lập mức tích cực (1) và khởi động timer ms

Mốc 20.16 ms: Bộ đếm timer_us đạt ngưỡng, FSM chuyển sang ST_MONITOR (current_state = 2). Ngõ ra led4_enout xác lập mức 1, đồng thời timer_ms khởi tạo ở mức 0 để bắt đầu chu kỳ đếm giám sát tWD mới



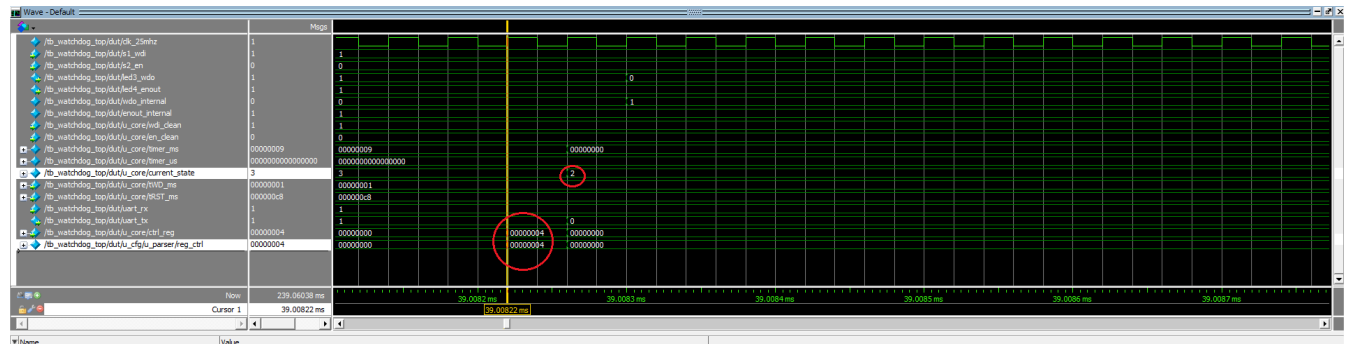
Hình 4.2.5. Tín hiệu Kick hợp lệ (sườn xuống) vượt qua mạch lọc nhiễu 20ms và reset thành công bộ đếm giám sát.

Tại mốc 22.16ms, nút nhấn S1 được kích hoạt ($s1_wdi = 0$). Tuy nhiên, bộ đếm $timer_ms$ không bị reset ngay lập tức. Hệ thống phải đợi tín hiệu đi qua (Debounce). Đến mốc 42.01 ms (sau ~20ms), tín hiệu wdi_clean mới tạo sườn xuống (Falling Edge) hợp lệ. Ngay lập tức, FSM nhận diện được cú Kick, ép bộ đếm $timer_ms$ đang ở giá trị 21 trở về 00000000 và bắt đầu đếm lại chu kỳ mới.



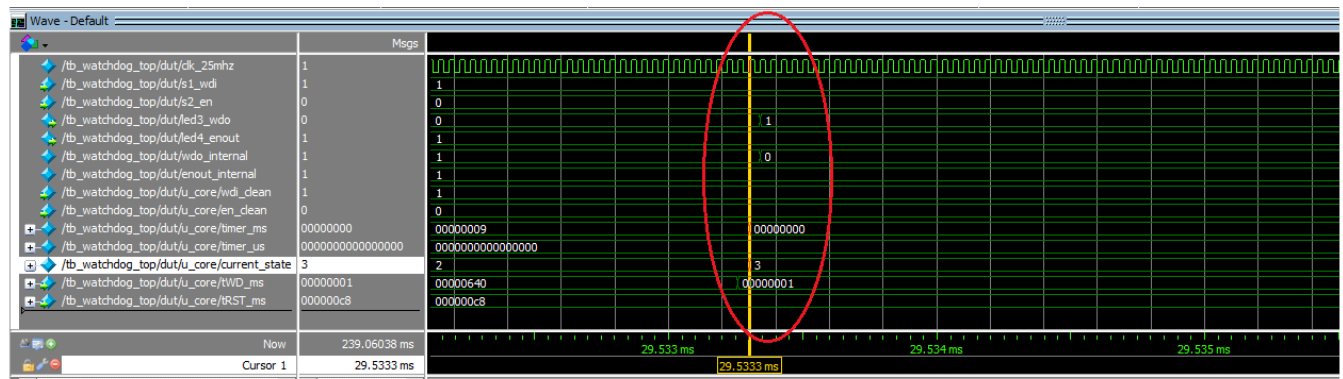
Hình 4.2.7. Mô phỏng hệ thống phát hiện Timeout và chuyển sang trạng thái lỗi (ST_FAULT) khi ngưỡng tWD_ms bị vượt qua

Tại mốc 29.533 ms, thanh ghi ngưỡng Timeout (tWD_ms) được chạy từ 0x0640 (1600ms) xuống 0x0001 (1ms). Vì bộ đếm $timer_ms$ lúc này đang có giá trị là 9 (tích lũy từ trước, lớn hơn ngưỡng 1ms mới), hệ thống lập tức xác nhận điều kiện Timeout. FSM chuyển ngay từ ST_MONITOR (2) sang ST_FAULT (3), ngõ ra báo lỗi $led3_wdo$ vọt lên mức 1, và bộ đếm $timer_ms$ được reset về 0 để chuẩn bị cho chu kỳ đếm thời gian Reset



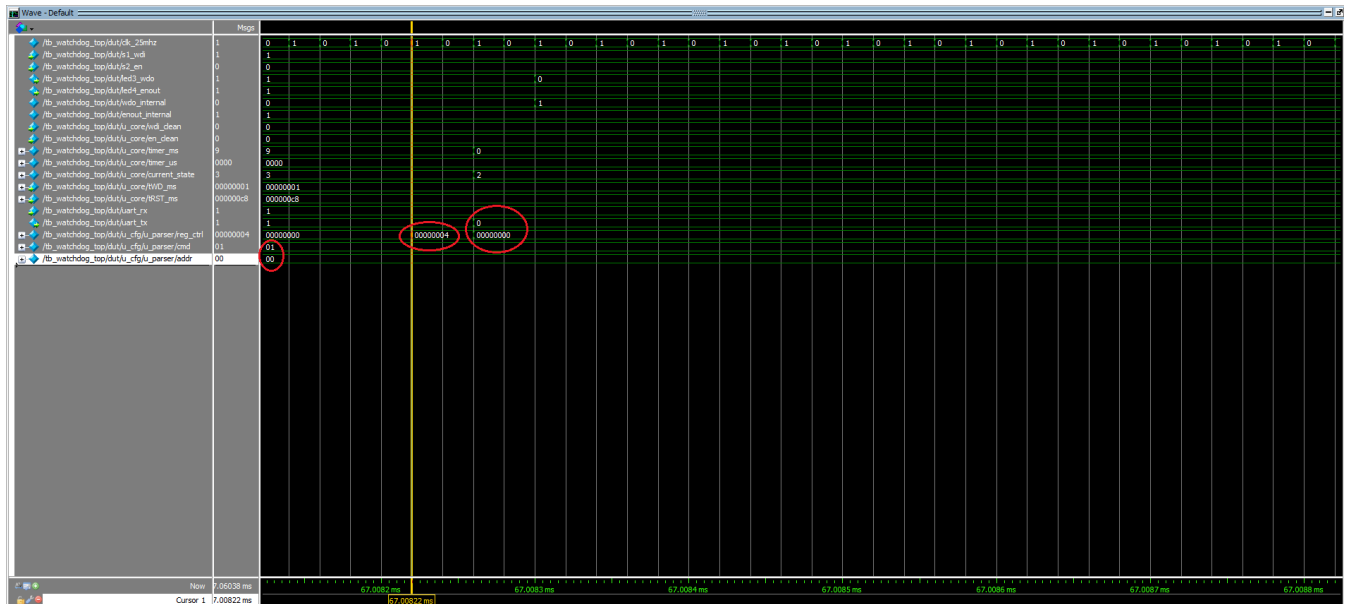
Hình 4.2.8. Chức năng Software Clear qua UART xóa trạng thái lỗi (ST_FAULT) ngay lập tức

Tại mốc 39.008 ms, hệ thống nhận thành công lệnh UART ghi vào thanh ghi điều khiển, làm ctrl_reg chuyển sang giá trị 0x04. Ngay tại chu kỳ xung nhịp tiếp theo, FSM đáp ứng tức thời bằng cách chuyển trạng thái từ ST_FAULT (3) về lại ST_MONITOR (2), đồng thời dập tắt ngõ ra báo lỗi WDO.



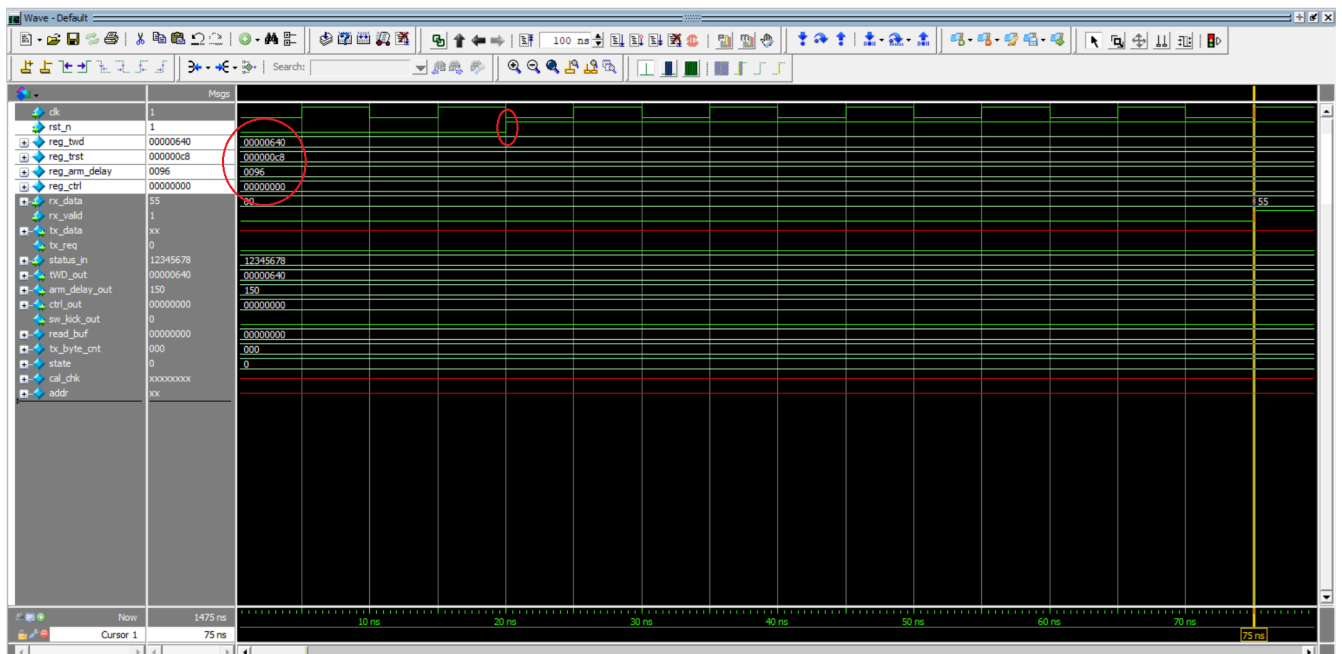
Hình 4.3. Tính chất phân cực của ngõ ra WDO và ENOUT.

Tại mốc 29.533 ms, khi hệ thống phát hiện lỗi và chuyển sang ST_FAULT (3), tín hiệu wdo_internal kéo xuống mức 0 (Active-low). Tuy nhiên, qua bộ đệm phân cực, tín hiệu vật lý led3_wdo được đảo logic lên mức 1 để kích sáng đèn LED báo hiệu. Tín hiệu led4_enout đồng bộ hoàn toàn với logic nội bộ, thỏa mãn yêu cầu xuất Push-pull tương thích phân cực.

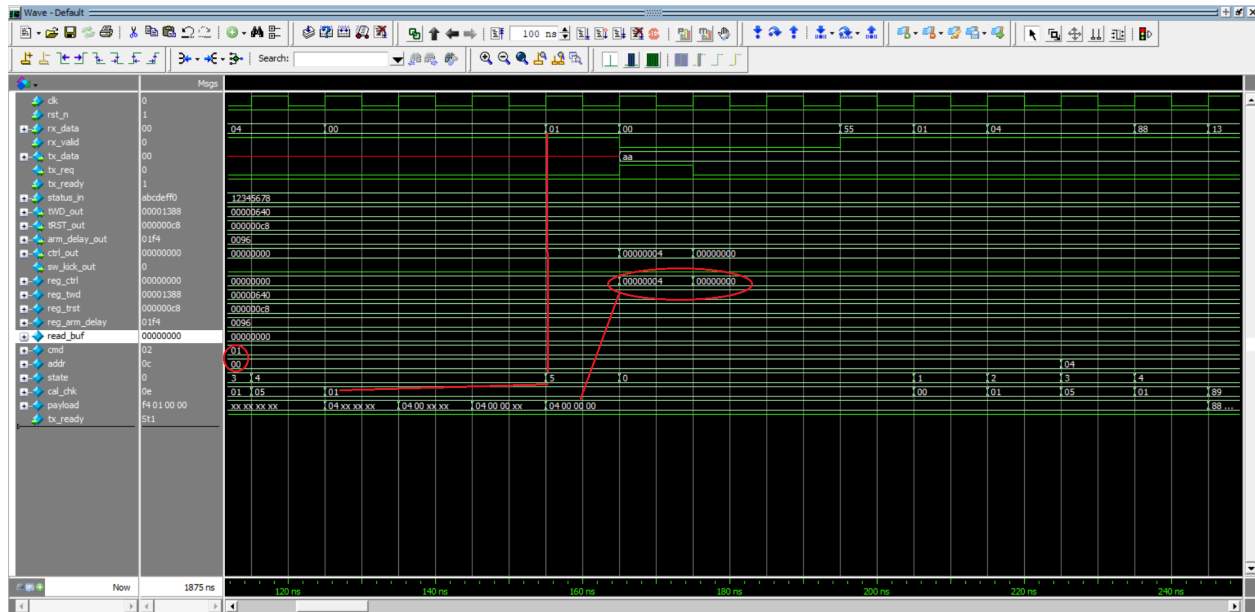


Dựa trên dạng sóng tại mốc 67.008 ms, thanh ghi reg_ctrl được cập nhật giá trị mới. Điều này cho thấy gói tin UART đầu vào đã được bộ Parser nhận diện đúng cấu trúc Frame và thỏa mãn điều kiện kiểm tra Checksum. Ngay sau chu kỳ cập nhật, ngõ ra uart_tx chuyển xuống mức 0 (Start bit) để bắt đầu truyền byte ACK (0xAA) phản hồi.

Dữ liệu 32-bit 0x00000004 được ghi vào đúng địa chỉ thanh ghi CTRL (0x00) qua biến addr. Thao tác ghi hoàn tất xác nhận lệnh WRITE_REG (0x01) qua biến cmd đã được hệ thống phân tích và thực thi

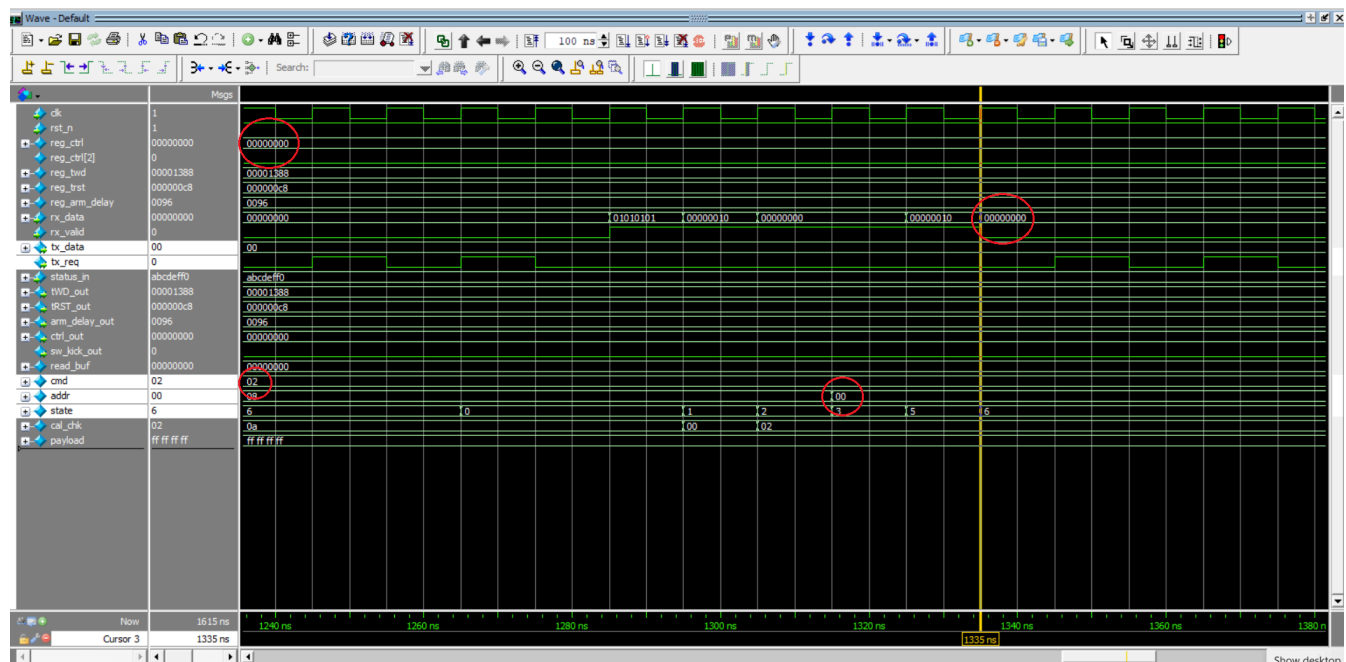


Hình 5.2.1: Ảnh waveform thể hiện quá trình khởi tạo hệ thống khi tín hiệu reset bất đồng bộ (rst_n) đang ở mức thấp và thời điểm giải phóng reset tại mốc 20ns. Các thanh ghi nội bộ của module `regfile_and_parser` được nạp các giá trị mặc định, cụ thể thanh ghi điều khiển `reg_ctrl` (32-bit) được đưa về giá trị 0. Thanh ghi cấu hình thời gian watchdog `reg_twd` (32-bit) nhận giá trị mặc định là 1600 (hiển thị dưới dạng Hex là 0x00000640), và thanh ghi thời gian reset hệ thống `reg_trst` (32-bit) khởi tạo ở giá trị 200 (0x000000C8). Đối với các tham số cấu hình khác, thanh ghi `reg_arm_delay` được thiết lập đúng độ rộng 16-bit với giá trị 150 (0x0096).



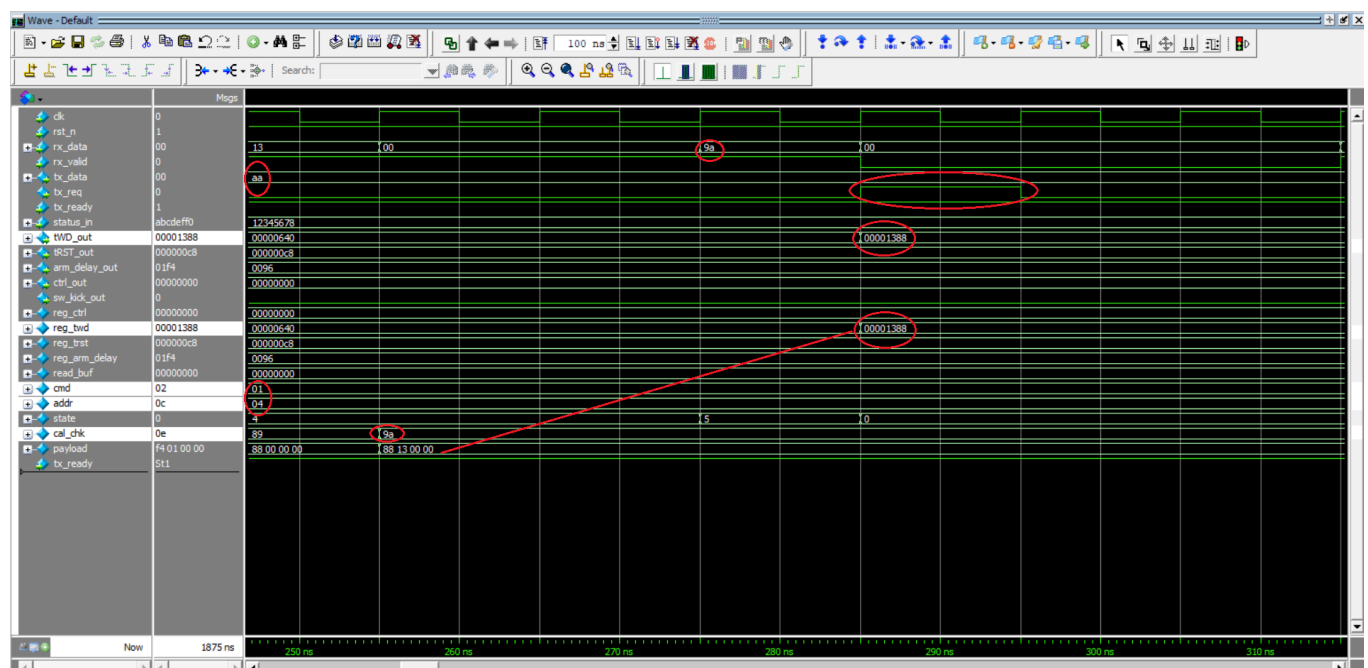
Hình 5.2.2: Tính chất write của thanh ghi CTRL.

`cmd=01`, `addr=00`, State sang `S_CHK` (5) check `rx_data=cal_chk`. Ngay sau đó, `reg_ctrl` chuyển từ 0 sang 4 (0x00000004) khi nhận các bit từ 0-4 của payload. Tại chu kỳ tiếp theo bit 2 của `reg_ctrl` tự động về 0 do logic xóa xung điều khiển ảo.

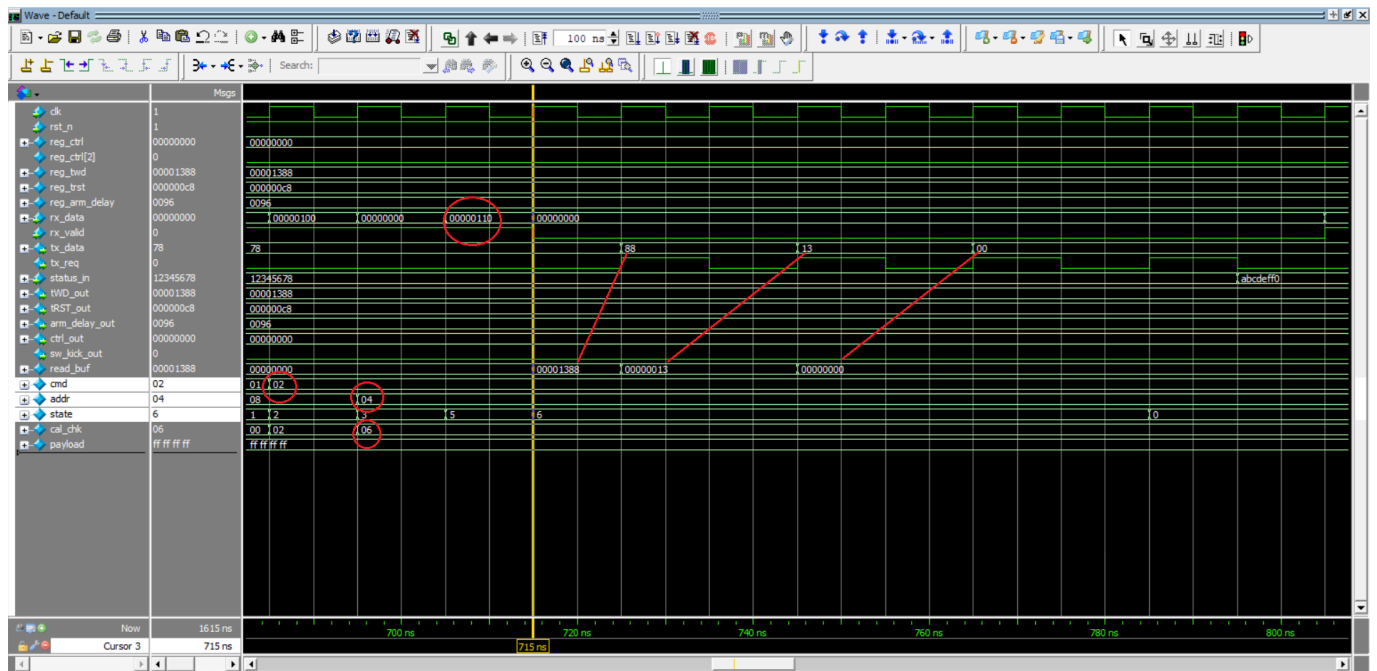


Hình 5.2.3: Tính chất read của thanh ghi CTRL

Tại đây, module nhận lệnh Read (cmd = 0x02) tại addr 0x00 . FSM tiếp tục xác thực dữ liệu, đến S_CHK (5) thì kiểm tra rx_data=cal_ckl, thực hiện nạp giá trị từ reg_ctrl vào bộ đệm và đẩy ra giao diện UART . Do bit 2 (W1C) đã tự động xóa về 0 từ trước đó, kết quả đọc về hiển thị trên đường tx_data là 0x00000000 thông qua 4 xung tx_req liên tiếp.

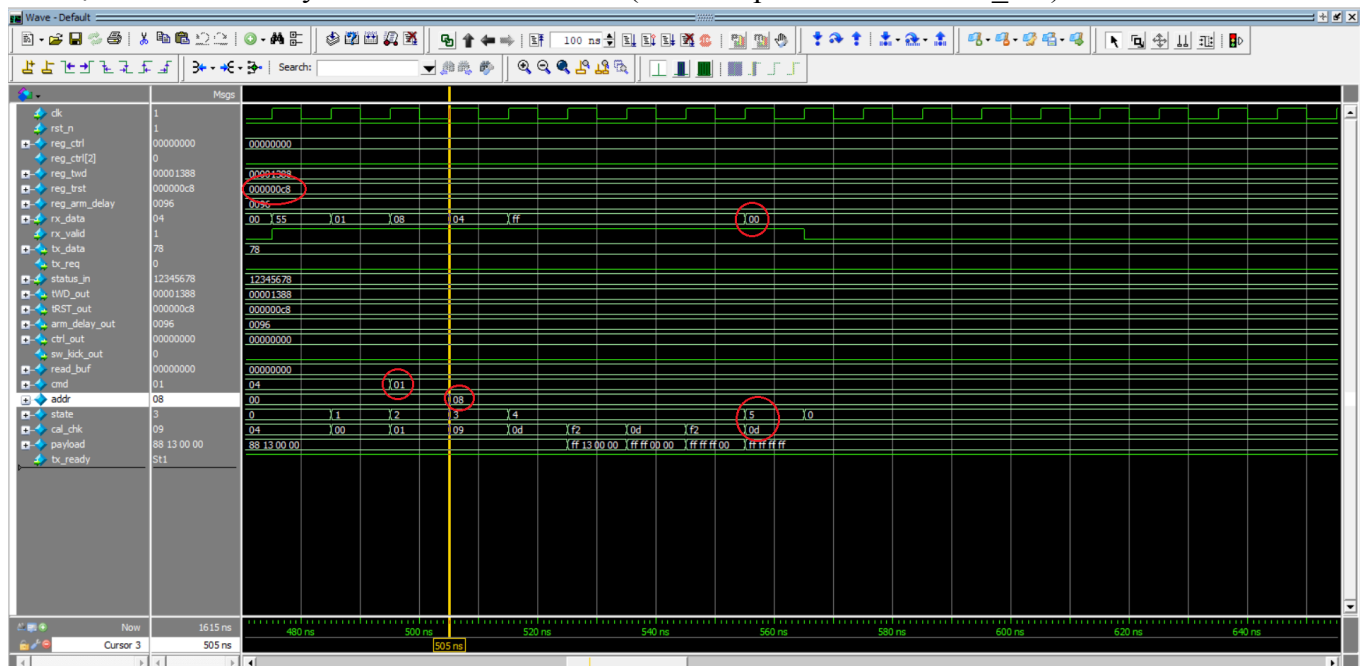


Hình 5.2.4: Tính chất write của thanh ghi tWD_ms
cmd=01, addr=04, FSM xác thực dữ liệu, đến S_CHK thì kiểm tra rx_data=cal_chk, tín hiệu reg_twd (và tWD_out) thay đổi từ giá trị mặc định 1600 (0x640) sang 5000 (0x1388) qua việc ghi các bit từ 0-4 của payload



Hình 5.2.5: Tính chất read của thanh ghi tWD_ms

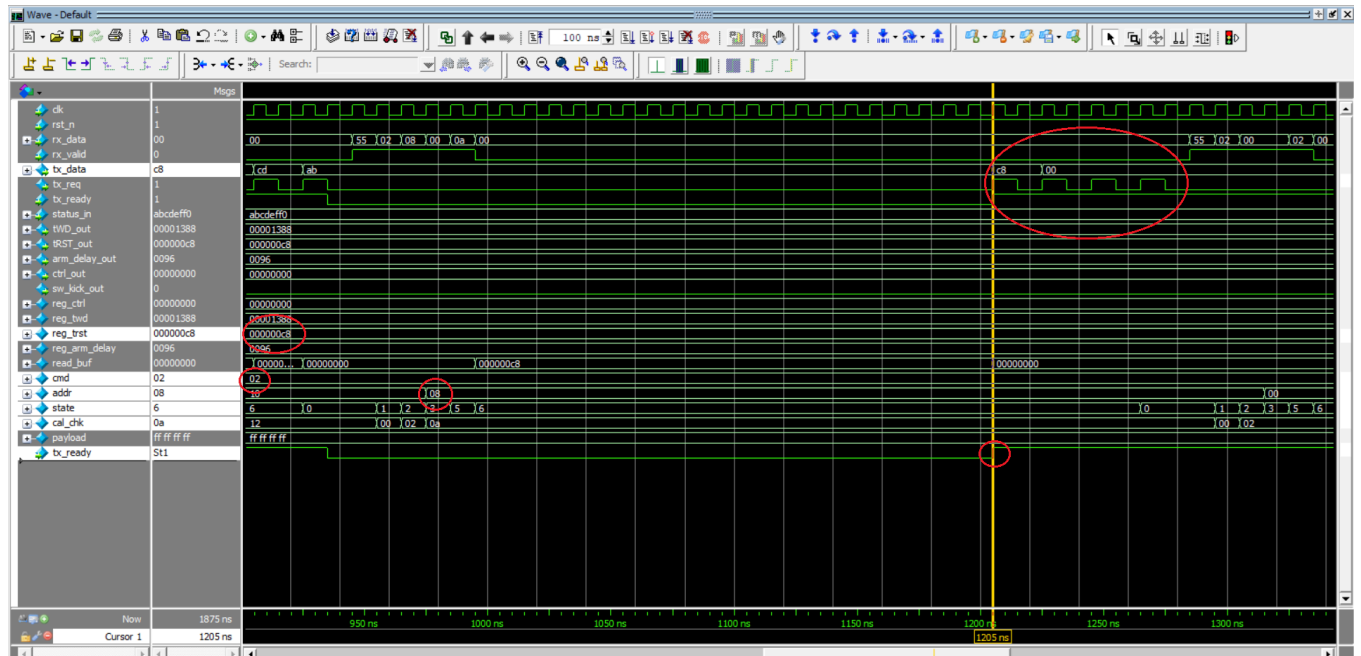
cmd=02, addr=04, FSM xác thực dữ liệu đến S_CHK (5) thì kiểm tra rx_data=cal_chk, tx_data lần lượt xuất ra chuỗi byte: 88 -> 13 -> 00 -> 00 (từ bit thấp đến cao của read_buf)



Hình 5.2.6: Tính chất write của thanh ghi tRST_ms

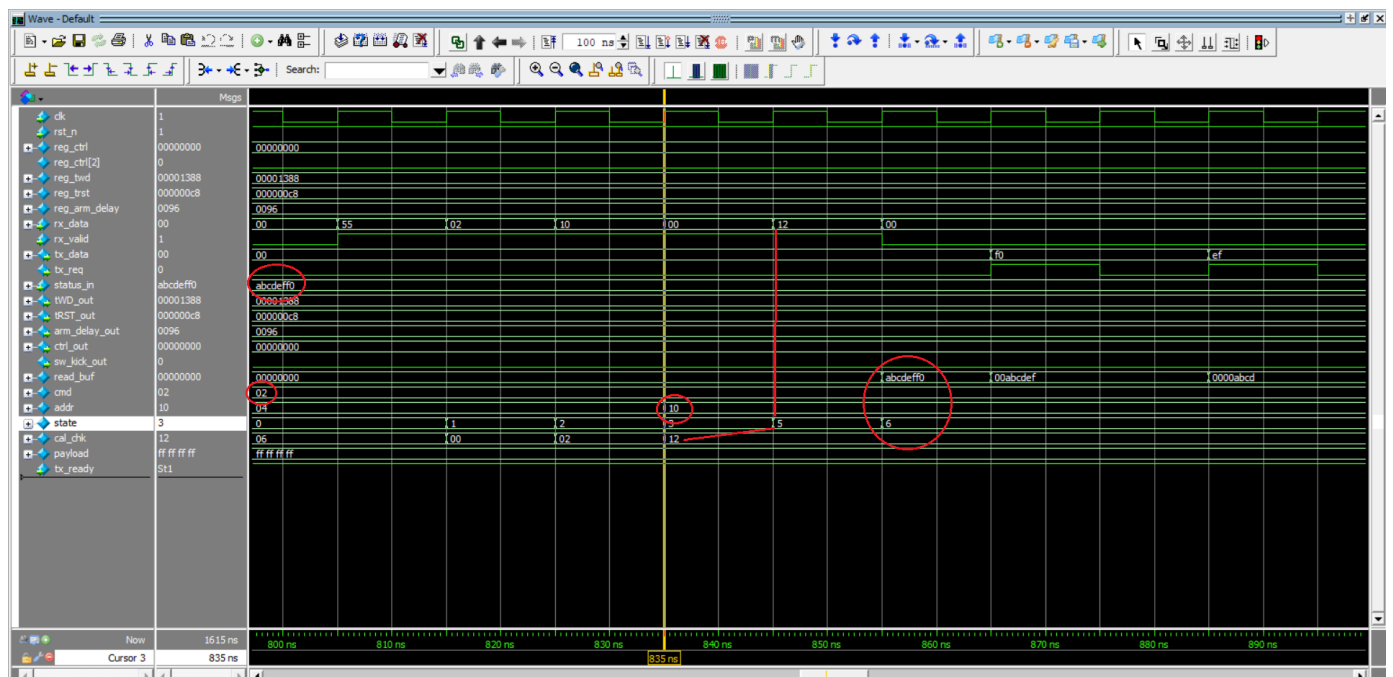
cmd=01, addr=08, FSM tiếp tục xác thực dữ liệu đến S_CHK thì kiểm tra rx_data khác cal_chk, điểm đặc biệt của ở đây là khả năng bảo vệ tính toàn vẹn của register file: khi khung tin gặp lỗi Checksum (giá trị thực tế không khớp với cal_chk), hệ thống lập tức hủy bỏ giao dịch, giữ

nguyên giá trị thanh ghi ở mức mặc định 200 (0xC8) -> khẳng định tính chất write của thanh ghi tRST_ms.



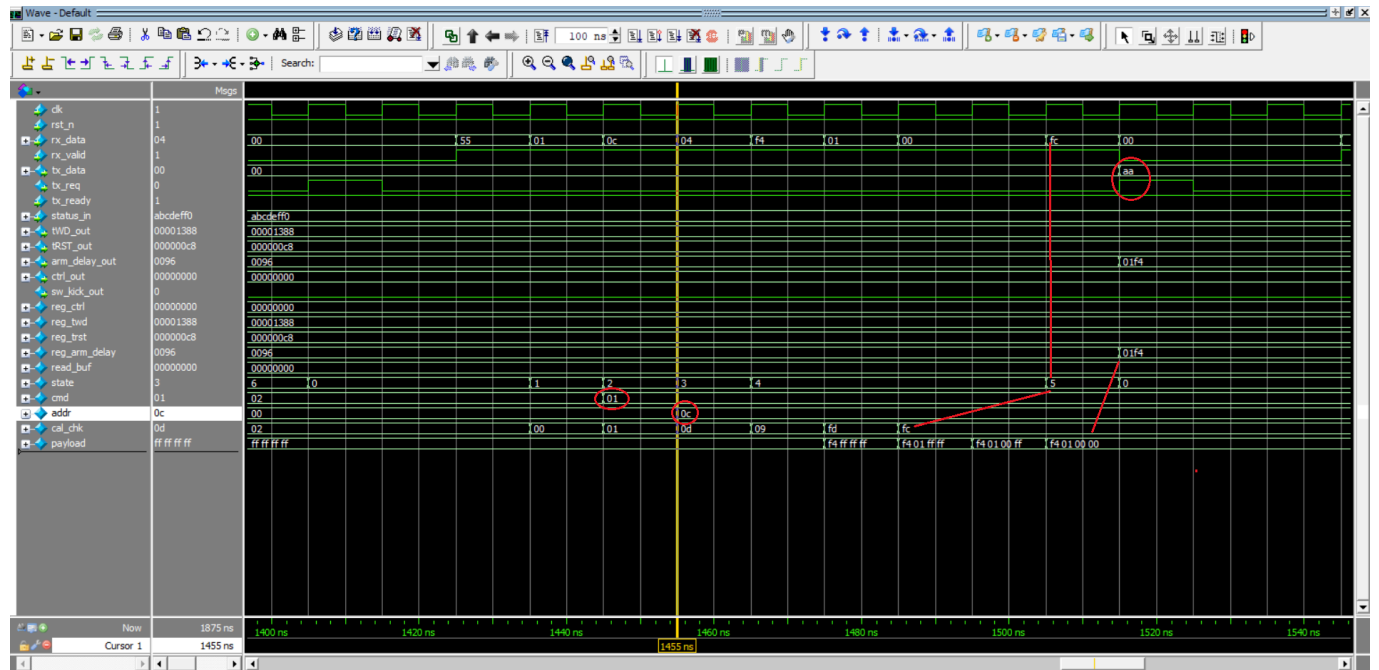
Hình 5.2.7: Tính chất read của thanh ghi tRST_ms

cmd=02, addr=08, FSM tiếp tục xác thực dữ liệu đến S_CHK thì kiểm tra rx_data=cal_chk, tx_ready được giải phóng về 1, tx_data bắt đầu xuất các byte của giá trị mặc định 200 (0xC8), chứng minh việc truy xuất dữ liệu từ thanh ghi 0x08 vẫn chính xác ngay cả khi UART đang bận.



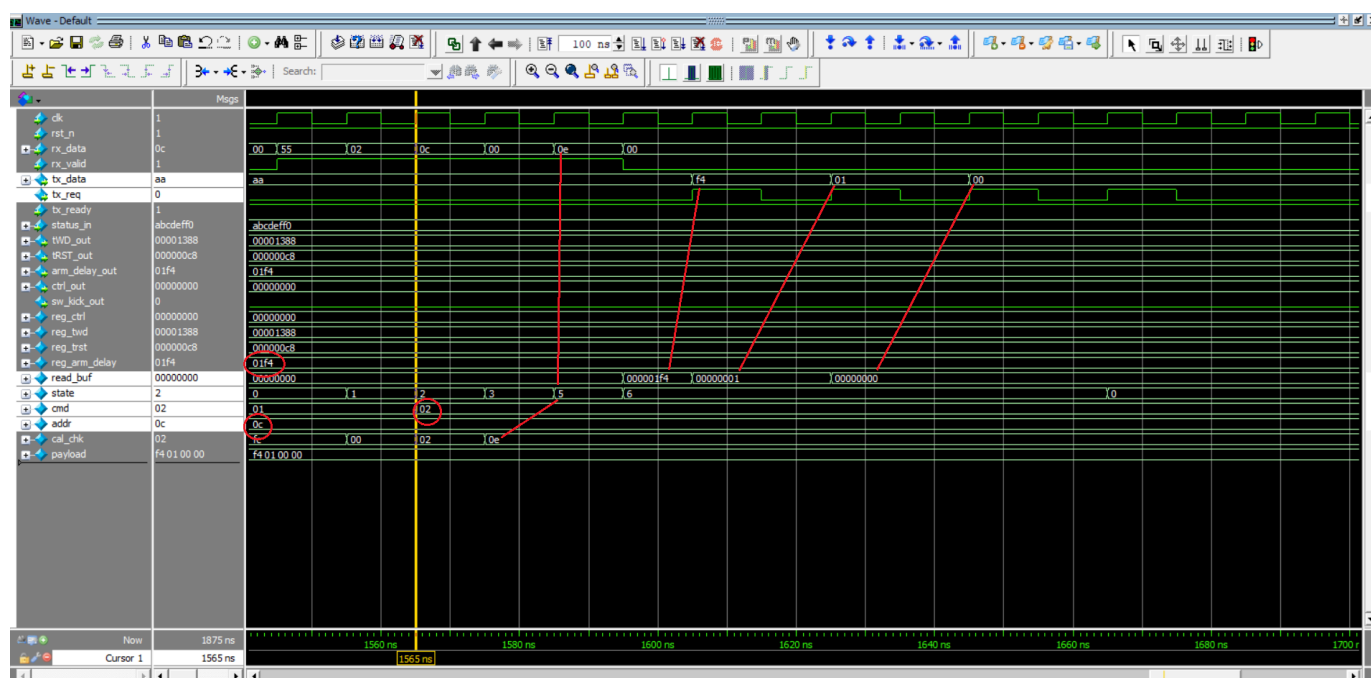
Hình 5.2.8: Tính chất read của thanh ghi STATUS

cmd=02, addr=10, FSM tiếp tục xác thực dữ liệu đến S_CHK thì kiểm tra rx_data=cal_chk sau đó read_buf <= status_in. giá trị abcdeff0 sẽ chính thức xuất hiện tại read_buf ở cạnh lên tiếp theo của clock, đồng thời với việc state chuyển sang mức S_TX_READ (6)

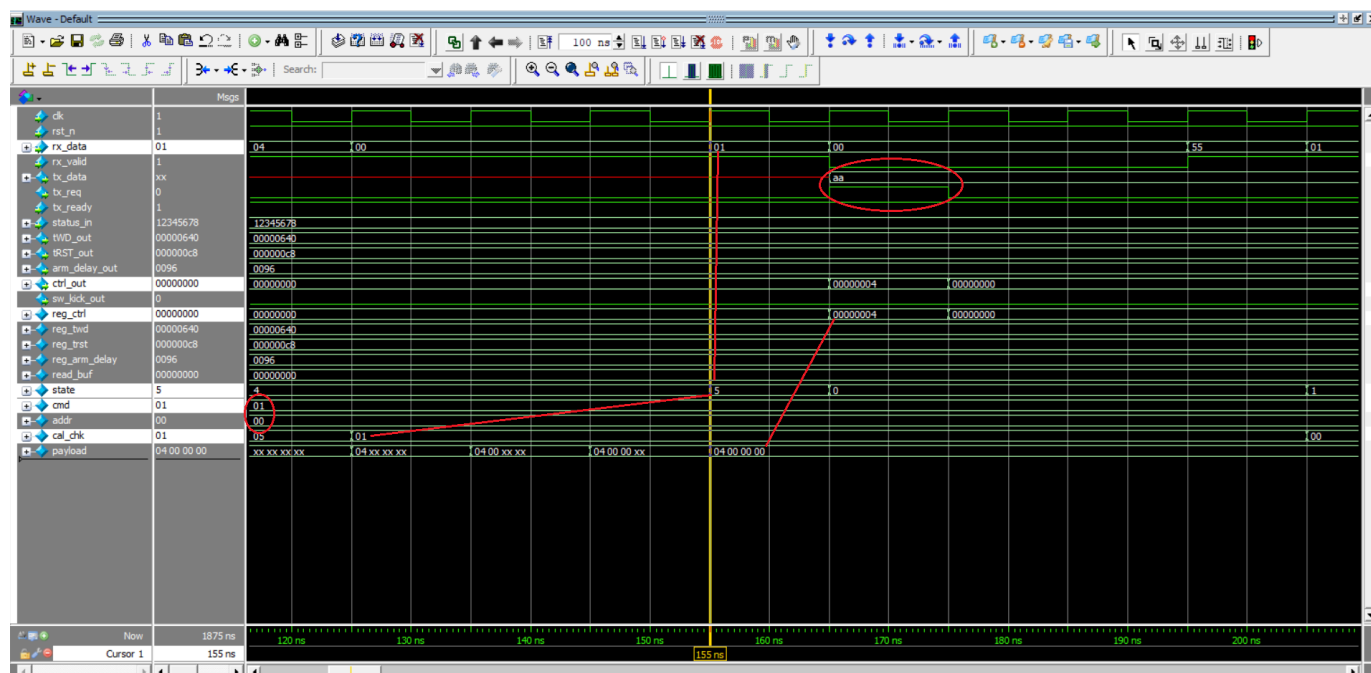


Hình 5.2.9: Tính chất write của thanh ghi arm_delay_us

cmd=01, addr=0c, FSM tiếp tục xác thực dữ liệu đến S_CHK thì kiểm tra rx_data=cal_chk, quan sát tín hiệu reg_arm_delay thấy giá trị chuyển từ 0096 (150) sang 01f4 (500) khi state kết thúc trạng thái S_CHK (lấy các bit từ 0-4 của payload). Tín hiệu tx_data xuất hiện aa và xuất hiện xung tx_req.



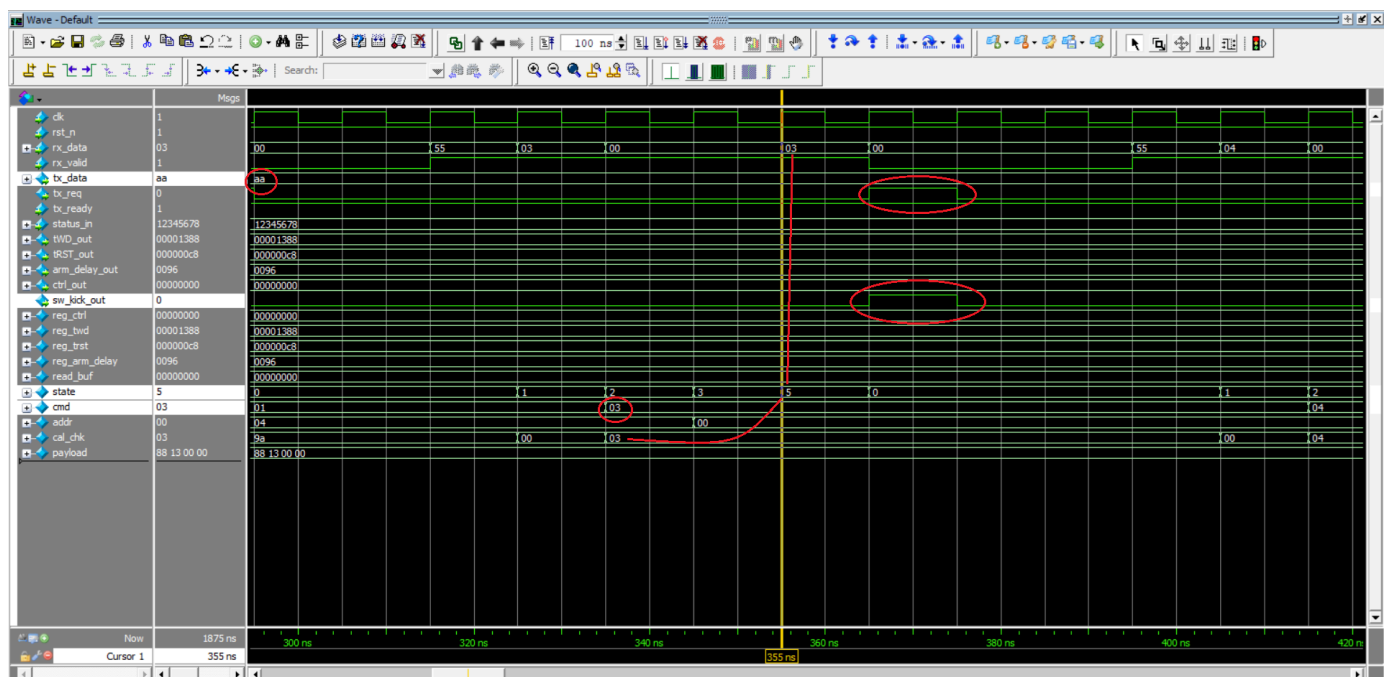
cmd=01, addr=0c, FSM tiếp tục xác thực dữ liệu đến S_CHK thì kiểm tra rx_data = cal_chk, reg_arm_delay<=payload[1:0], tín hiệu read_buf nạp giá trị 000001f4 của reg_arm_delay, tín hiệu tx_data lần lượt xuất hiện chuỗi byte: f4 -> 01 -> 00 -> 00 (thứ tự byte thấp trước)



Hình 5.3.1: Lệnh 0x01 WRITE_REG

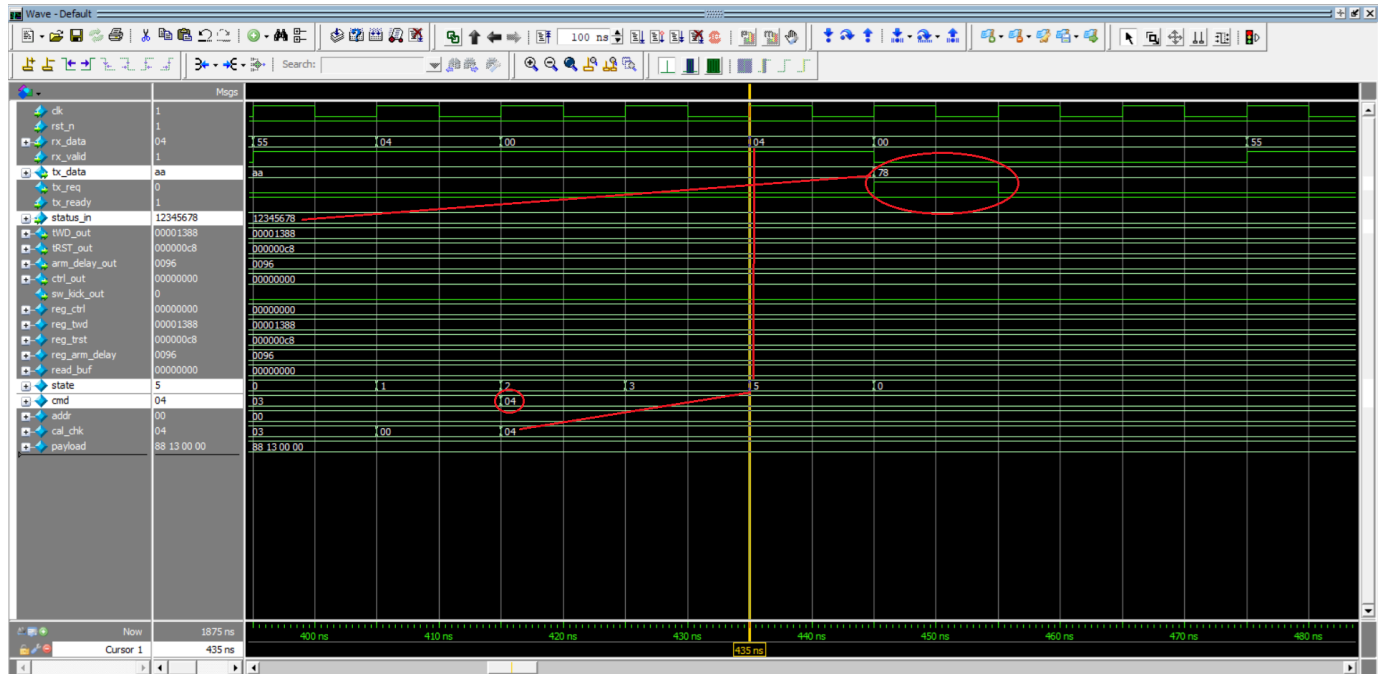
Thời điểm 155 ns, cmd=01, FSM xác thực dữ liệu đầu vào đến S_CHK thấy rx_data == cal chk-> module sử dụng địa chỉ addr để xác định thanh ghi đích và nạp dữ liệu từ mảng

cmd=02, addr=04, FSM xác thực dữ liệu đầu vào đến S_CHK thấy rx_data == cal_chk, dữ liệu từ thanh ghi được chỉ định bởi addr sẽ được nạp vào read_buf, FSM chuyển sang trạng thái S_TX_READ, để thực hiện gửi 4 byte qua UART. Dữ liệu thanh ghi được gửi theo thứ tự từ thấp đến cao với 4 xung tx_req liên tiếp



Hình 5.3.3: Lệnh 0x03 KICK

cmd=03, FSM xác thực dữ liệu đầu vào đến S_CHK thấy rx_data == cal_chk, module kích hoạt tín hiệu sw_kick_out lên mức cao trong đúng 1 chu kỳ clock và trả về ACK tx_data=aa, tín hiệu tx_req cũng lên cao 1 chu kỳ clock



Hình 5.3.4: Lệnh 0x04 GET_STATUS

cmd=04, FSM xác thực dữ liệu đầu vào đến S_CHK thấy rx_data == cal_chk, gán trực tiếp 8-bit thấp của status_in vào tx_data và đi kèm 1 xung tx_req duy nhất.